Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040038
Nama	Derania Safa Zaina
Kelas	F
Link Github Repository	[isi link repository GitHub setelah di-push]

Bagian 1. Screenshot Semua Kode

Proyek: pertemuan2 (Vite + React + TypeScript). Semua kode berada di folder src/. src/index.css dikosongkan sesuai Langkah 1.

src/main.tsx – titik masuk (tidak diubah)

src/main.tsx

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.css – Langkah 1 – gaya tampilan (tema coklat) + perbaikan tata letak form di bagian akhir

src/App.css (bagian 1/2)

```css
/* src/App.css - tema coklat tua & coklat muda */
:root {
  --coklat-tua: #4E342E;     /* espresso */
  --coklat-sedang: #6D4C41;
  --coklat-muda: #A1887F;
  --krem: #F6EEE6;           /* latar halaman */
  --krem-muda: #FBF7F2;      /* latar kartu / item */
  --garis: #E2D5C9;
  --teks: #3E2C25;
  --teks-lembut: #8A7468;
  --hijau: #5E7F3B;          /* pemasukan */
  --merah: #B23A2B;          /* pengeluaran / hapus */
}

body { background: var(--krem); }

.wadah { max-width: 640px; margin: 40px auto; padding: 0 20px 40px; font-family: "Segoe UI", Arial, sans-serif; color: var(--teks); }
.judul { font-family: Georgia, "Times New Roman", serif; color: var(--coklat-tua); margin-bottom: 4px; letter-spacing: 0.3px; }
.subjudul { color: var(--teks-lembut); margin-top: 0; margin-bottom: 24px; }

.kartu { background: var(--krem-muda); border: 1px solid var(--garis); border-top: 4px solid var(--coklat-muda); border-radius: 14px; padding: 20px; margin-bottom: 20px; box-shadow: 0 6px 16px -6px rgba(78, 52, 46, 0.22); }
.kartu h2 { margin-top: 0; font-family: Georgia, "Times New Roman", serif; font-size: 18px; color: var(--coklat-tua); }

.baris { display: flex; gap: 8px; }
.input { flex: 1; padding: 10px 12px; border: 1px solid var(--garis); border-radius: 8px; font-size: 14px; background: #FFFFFF; color: var(--teks); }
.input:focus, .pilih:focus { outline: 2px solid var(--coklat-muda); border-color: var(--coklat-muda); }
.input::placeholder { color: #B8A89D; }

.tombol { padding: 10px 16px; border: none; border-radius: 8px; background: var(--coklat-tua); color: #FFFFFF; font-weight: bold; cursor: pointer; transition: background 0.2s, transform 0.1s; }
.tombol:hover:not(:disabled) { background: var(--coklat-sedang); }
.tombol:active:not(:disabled) { transform: translateY(1px); }
.tombol:disabled { background: #CDBFB5; cursor: not-allowed; }
.tombol-merah { background: var(--merah); }
.tombol-merah:hover:not(:disabled) { background: #93301F; }
.tombol-kecil { padding: 6px 10px; font-size: 12px; }

.daftar { list-style: none; padding: 0; margin: 0; }
.item { display: flex; align-items: center; gap: 10px; padding: 10px 0; border-bottom: 1px solid var(--garis); }
.item:last-child { border-bottom: none; }
.item span { flex: 1; }
.item input[type="checkbox"] { accent-color: var(--coklat-tua); width: 16px; height: 16px; }
.selesai { text-decoration: line-through; color: #B8A89D; }
.kosong { color: var(--teks-lembut); text-align: center; font-style: italic; padding: 8px 0; }
.galat { color: var(--merah); font-size: 13px; margin: 6px 0 0; }
.ringkasan { font-size: 14px; color: var(--coklat-sedang); }
```

src/App.css (bagian 2/2)

```
/* Tugas 1-3 */
.tab { display: flex; gap: 8px; margin-bottom: 20px; padding: 6px; background: #EADFD3; border-radius: 12px; }
.tab .tombol-filter { flex: 1; }
.tombol-filter { padding: 8px 14px; border: 1px solid transparent; border-radius: 8px; background: transparent; color: var(--coklat-sedang); cursor: pointer; font-size: 13px; transition: background 0.2s; }
.tombol-filter:hover { background: #F3E9DE; }
.tombol-filter.aktif { background: var(--coklat-tua); border-color: var(--coklat-tua); color: #FFFFFF; font-weight: bold; }
.filter { display: flex; gap: 8px; margin: 10px 0 12px; }
.filter .tombol-filter { background: #FFFFFF; border-color: var(--garis); }
.filter .tombol-filter.aktif { background: var(--coklat-tua); border-color: var(--coklat-tua); }
.aksi { margin-top: 14px; }
.pemasukan { color: var(--hijau); font-weight: bold; }
.pengeluaran { color: var(--merah); font-weight: bold; }
.saldo { font-family: Georgia, "Times New Roman", serif; font-size: 30px; font-weight: bold; margin: 4px 0 0; }
.rekap { display: flex; gap: 16px; font-size: 14px; margin-top: 6px; }
.pilih { padding: 10px; border: 1px solid var(--garis); border-radius: 8px; font-size: 14px; background: #FFFFFF; color: var(--teks); }
.form-kolom { display: grid; gap: 8px; }

/* Perbaikan tata letak form transaksi: gaya .catatan-keuangan membuat input dan select selebar 100%,
   sehingga kolom Nominal mengecil. Aturan ini mengembalikan lebarnya. */
.form-kolom .baris select { width: auto; }
.form-kolom .baris .input { width: auto; min-width: 0; }
```

src/Kartu.tsx – Langkah 2 – komponen pembungkus (props children)

src/Kartu.tsx

```
// src/Kartu.tsx
import type { ReactNode } from "react";

type KartuProps = {
  judul: string;
  children: ReactNode; // isi bebas di antara <Kartu> dan </Kartu>
};

// Komponen pembungkus: judul dikirim lewat props, isi dikirim lewat children
function Kartu({ judul, children }: KartuProps) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

src/Penghitung.tsx (versi keliru, Langkah 3a)

src/Penghitung.tsx (versi keliru, Langkah 3a)

```
// src/Penghitung.tsx (VERSI KELIRU - untuk eksperimen)
import Kartu from "./Kartu";

function Penghitung() {
  let jumlah = 0; // variabel biasa

  const tambah = () => {
    jumlah = jumlah + 1;
    console.log("Nilai jumlah sekarang:", jumlah);
  };

  return (
    <Kartu judul="Penghitung Klik">
      <p>Tombol sudah diklik {jumlah} kali</p>
      <button className="tombol" onClick={tambah}>Klik Saya</button>
    </Kartu>
  );
}

export default Penghitung;
```

src/Penghitung.tsx (versi benar, Langkah 3b)

src/Penghitung.tsx (versi benar, Langkah 3b)

```tsx
// src/Penghitung.tsx (VERSI BENAR)
import { useState } from "react";
import Kartu from "./Kartu";

function Penghitung() {
  // jumlah = nilai saat ini, setJumlah = fungsi untuk mengubahnya, 0 = nilai awal
  const [jumlah, setJumlah] = useState(0);

  const tambah = () => {
    setJumlah(jumlah + 1);
  };

  const ulangi = () => {
    setJumlah(0);
  };

  return (
    <Kartu judul="Penghitung Klik">
      <p>Tombol sudah diklik {jumlah} kali</p>

      {/* Conditional rendering: pesan ini hanya muncul jika jumlah >= 5 */}
      {jumlah >= 5 && <p>Wah, kamu rajin sekali mengklik!</p>}

      <div className="baris">
        <button className="tombol" onClick={tambah}>Klik Saya</button>
        <button className="tombol tombol-merah" onClick={ulangi}>Ulangi</button>
      </div>
    </Kartu>
  );
}

export default Penghitung;
```

src/FormTugas.tsx – Langkah 4 – formulir terkendali

src/FormTugas.tsx

```tsx
// src/FormTugas.tsx
import { useState } from "react";
import type { FormEvent } from "react";

type FormTugasProps = {
  onTambah: (judul: string) => void; // fungsi "pelapor" dari induk
};

function FormTugas({ onTambah }: FormTugasProps) {
  // State lokal untuk menampung isi input
  const [teks, setTeks] = useState("");

  // Validasi sederhana: tidak boleh kosong dan maksimal 50 karakter
  const terlaluPanjang = teks.length > 50;
  const tidakValid = teks.trim() === "" || terlaluPanjang;

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault(); // mencegah halaman reload saat formulir dikirim
    if (tidakValid) return; // hentikan jika input tidak valid

    onTambah(teks.trim()); // "melapor" ke komponen induk
    setTeks(""); // kosongkan kembali input
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="baris">
        <input
          className="input"
          type="text"
          placeholder="Contoh: Kerjakan laporan Modul 2"
          value={teks}
          onChange={(event) => setTeks(event.target.value)}
        />
        <button className="tombol" type="submit" disabled={tidakValid}>
          Tambah
        </button>
      </div>
      {terlaluPanjang && <p className="galat">Judul tugas maksimal 50 karakter.</p>}
    </form>
  );
}

export default FormTugas;
```

src/ItemTugas.tsx – Langkah 5 – satu baris tugas

src/ItemTugas.tsx

```tsx
// src/ItemTugas.tsx
export type Tugas = {
  id: number;
  judul: string;
  selesai: boolean;
};

type ItemTugasProps = {
  tugas: Tugas;
  onUbahStatus: (id: number) => void;
  onHapus: (id: number) => void;
};

function ItemTugas({ tugas, onUbahStatus, onHapus }: ItemTugasProps) {
  return (
    <li className="item">
      <input
        type="checkbox"
        checked={tugas.selesai}
        onChange={() => onUbahStatus(tugas.id)}
      />

      {/* className dinamis: dicoret jika tugas sudah selesai */}
      <span className={tugas.selesai ? "selesai" : ""}>{tugas.judul}</span>

      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(tugas.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTugas;
```

src/DaftarTugas.tsx – Langkah 6–7 + Tugas 1 & 2 – komponen induk (memuat gaya tema hijau sendiri)

src/DaftarTugas.tsx (bagian 1/5)

```tsx
// src/DaftarTugas.tsx (komponen induk aplikasi Daftar Tugas Praktikum)
import { useState, useEffect } from "react";
import Kartu from "./Kartu";
import Penghitung from "./Penghitung";
import FormTugas from "./FormTugas";
import ItemTugas from "./ItemTugas";
import type { Tugas } from "./ItemTugas";

type Filter = "semua" | "belum" | "selesai";

const tugasAwal: Tugas[] = [
  { id: 1, judul: "Install Node.js dan Vite", selesai: true },
  { id: 2, judul: "Pelajari useState", selesai: false },
];

function DaftarTugas() {
  // 1. State utama: coba ambil dari localStorage, jika kosong pakai data contoh
  const [daftarTugas, setDaftarTugas] = useState<Tugas[]>(() => {
    try {
      const dataTersimpan = localStorage.getItem("daftar-tugas");
      if (dataTersimpan) {
        return JSON.parse(dataTersimpan) as Tugas[]; // ubah teks JSON kembali menjadi array
      }
    } catch {
      // isi localStorage rusak: abaikan dan pakai data contoh
    }
    return tugasAwal;
  });

  // Tugas 1: state untuk pilihan filter
  const [filter, setFilter] = useState<Filter>("semua");

  // Efek A: dijalankan SEKALI saat komponen pertama kali tampil
  useEffect(() => {
    console.log("Aplikasi Daftar Tugas siap digunakan!");
  }, []);

  // Efek B: perbarui judul tab peramban setiap daftarTugas berubah
  useEffect(() => {
    const sisa = daftarTugas.filter((tugas) => !tugas.selesai).length;
    document.title = `(${sisa}) Tugas Belum Selesai`;
  }, [daftarTugas]);

  // Efek C: simpan daftarTugas ke localStorage setiap kali berubah
  useEffect(() => {
    localStorage.setItem("daftar-tugas", JSON.stringify(daftarTugas));
  }, [daftarTugas]);

  // 2. Menambah tugas: buat array BARU berisi tugas lama + tugas baru
  const tambahTugas = (judul: string) => {
    const tugasBaru: Tugas = { id: Date.now(), judul: judul, selesai: false };
    setDaftarTugas([...daftarTugas, tugasBaru]);
  };

  // 3. Mengubah status selesai: salin tugas yang dicentang, biarkan yang lain
  const ubahStatus = (id: number) => {
    const daftarBaru = daftarTugas.map((tugas) => {
      if (tugas.id === id) {
        return { ...tugas, selesai: !tugas.selesai }; // salinan dengan status dibalik
      }
```

src/DaftarTugas.tsx (bagian 2/5)

```
      return tugas; // tugas lain tidak berubah
    });
    setDaftarTugas(daftarBaru);
  };

  // 4. Menghapus tugas: ambil semua tugas KECUALI yang id-nya sama
  const hapusTugas = (id: number) => {
    setDaftarTugas(daftarTugas.filter((tugas) => tugas.id !== id));
  };

  // Tugas 2: hapus semua tugas yang berstatus selesai
  const hapusSemuaSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas) => !tugas.selesai));
  };

  // 5. Nilai turunan: dihitung dari state, tidak perlu state baru
  const jumlahSelesai = daftarTugas.filter((tugas) => tugas.selesai).length;

  // Tugas 1: saring daftar sesuai filter SEBELUM di-map
  const tugasTampil = daftarTugas.filter((tugas) => {
    if (filter === "belum") return !tugas.selesai;
    if (filter === "selesai") return tugas.selesai;
    return true; // "semua"
  });

  const pilihanFilter: { nilai: Filter; label: string }[] = [
    { nilai: "semua", label: "Semua" },
    { nilai: "belum", label: "Belum Selesai" },
    { nilai: "selesai", label: "Selesai" },
  ];

  return (
    <main className="daftar-tugas-app">
      <style>{`
        .daftar-tugas-app {
          --emerald-900: #064e3b;
          --emerald-700: #047857;
          --emerald-600: #059669;
          --emerald-100: #d1fae5;
          --green-50: #f0fdf4;
          min-height: 100vh;
          box-sizing: border-box;
          padding: clamp(24px, 5vw, 56px) 20px;
          background: radial-gradient(ellipse at top, #dcfce7 0%, #f0fdf4 42%, #f8fafc 100%);
          color: #1f2937;
          font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
serif;
        }
        .daftar-tugas-app * { box-sizing: border-box; }
        .daftar-tugas-app > .konten-tugas { width: min(100%, 760px); margin: 0 auto; }
        .daftar-tugas-app .judul {
          margin: 0;
          color: var(--emerald-900);
          font-size: clamp(2rem, 5vw, 2.8rem);
          font-weight: 800;
          letter-spacing: -0.045em;
          text-align: center;
        }
        .daftar-tugas-app .subjudul {
          margin: 10px 0 28px;
          color: #4b7564;
```

src/DaftarTugas.tsx (bagian 3/5)

```
          line-height: 1.6;
          text-align: center;
        }
        .daftar-tugas-app .area-penghitung,
        .daftar-tugas-app .panel-tugas {
          margin: 18px 0;
          padding: clamp(20px, 4vw, 30px);
          border: 1px solid #bbf7d0;
          border-radius: 20px;
          background: rgba(255, 255, 255, 0.94);
          box-shadow: 0 12px 32px rgba(6, 78, 59, 0.08);
        }
        .daftar-tugas-app .panel-tugas h2 {
          margin: 0 0 18px;
          color: var(--emerald-900);
          font-size: 1.25rem;
        }
        .daftar-tugas-app .ringkasan {
          display: inline-block;
          margin: 0 0 18px;
          padding: 8px 13px;
          border-radius: 999px;
          background: var(--green-50);
          color: var(--emerald-700);
          font-size: 0.92rem;
          font-weight: 700;
        }
        .daftar-tugas-app .filter { display: flex; flex-wrap: wrap; gap: 9px; margin-bottom: 18px; }
        .daftar-tugas-app button {
          border: 1px solid #a7f3d0;
          border-radius: 10px;
          background: #ecfdf5;
          color: var(--emerald-700);
          cursor: pointer;
          font: inherit;
          font-weight: 650;
          transition: background 160ms ease, color 160ms ease, transform 160ms ease, box-shadow 160ms ease;
        }
        .daftar-tugas-app button:hover { transform: translateY(-1px); box-shadow: 0 4px 12px rgba(5, 150, 105,
0.14); }
        .daftar-tugas-app button:focus-visible { outline: 3px solid #6ee7b7; outline-offset: 3px; }
        .daftar-tugas-app .tombol-filter { padding: 9px 14px; }
        .daftar-tugas-app .tombol-filter.aktif,
        .daftar-tugas-app .tombol:not(.tombol-merah) {
          border-color: var(--emerald-600);
          background: var(--emerald-600);
          color: white;
        }
        .daftar-tugas-app .daftar { display: grid; gap: 10px; margin: 0; padding: 0; list-style: none; }
        .daftar-tugas-app .kosong {
          margin: 0;
          padding: 20px;
          border: 1px dashed #86efac;
          border-radius: 14px;
          background: var(--green-50);
          color: #4b7564;
          text-align: center;
        }
        .daftar-tugas-app .aksi { margin-top: 20px; }
        .daftar-tugas-app .tombol-merah { padding: 10px 15px; border-color: #059669; background: #059669;
color: white; }
        .daftar-tugas-app .tombol-merah:hover { background: #047857; }
```

src/DaftarTugas.tsx (bagian 4/5)

```
        @media (max-width: 520px) {
          .daftar-tugas-app { padding-right: 14px; padding-left: 14px; }
          .daftar-tugas-app .filter { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); }
          .daftar-tugas-app .tombol-filter { padding: 9px 6px; font-size: 0.85rem; }
        }
      `}</style>
      <div className="konten-tugas">
        <h1 className="judul">Daftar Tugas Praktikum</h1>
        <p className="subjudul">Laboratorium Informatika - Modul 2: React Lanjutan</p>

        <div className="area-penghitung"><Penghitung /></div>

        <section className="panel-tugas">
          <Kartu judul="Tambah Tugas Baru">
            <FormTugas onTambah={tambahTugas} />
          </Kartu>
        </section>

        <section className="panel-tugas">
          <Kartu judul="Daftar Tugas">
        <p className="ringkasan">
          {jumlahSelesai} dari {daftarTugas.length} tugas selesai
        </p>

        {/* Tugas 1: tiga tombol filter, yang aktif tampil berbeda */}
        <div className="filter">
          {pilihanFilter.map((p) => (
            <button
              key={p.nilai}
              className={`tombol-filter ${filter === p.nilai ? "aktif" : ""}`}
              onClick={() => setFilter(p.nilai)}
            >
              {p.label}
            </button>
          ))}
        </div>

        {/* Conditional rendering dengan ternary: tampilkan pesan jika kosong */}
        {daftarTugas.length === 0 ? (
          <p className="kosong">Belum ada tugas. Yuk tambahkan tugas pertamamu!</p>
        ) : tugasTampil.length === 0 ? (
          <p className="kosong">Tidak ada tugas pada filter ini.</p>
        ) : (
          <ul className="daftar">
            {tugasTampil.map((tugas) => (
              <ItemTugas
                key={tugas.id}
                tugas={tugas}
                onUbahStatus={ubahStatus}
                onHapus={hapusTugas}
              />
            ))}
          </ul>
        )}

        {/* Tugas 2: tombol hanya muncul jika ada minimal satu tugas selesai */}
        {jumlahSelesai > 0 && (
          <div className="aksi">
            <button className="tombol tombol-merah" onClick={hapusSemuaSelesai}>
              Hapus Semua yang Selesai
```

src/DaftarTugas.tsx (bagian 5/5)

```
            </button>
          </div>
        )}
          </Kartu>
        </section>
      </div>
    </main>
  );
}

export default DaftarTugas;
```

src/App.tsx – kerangka: pindah halaman Daftar Tugas / Catatan Keuangan

src/App.tsx

```tsx
// src/App.tsx (kerangka: memilih halaman yang tampil)
import { useState } from "react";
import "./App.css";
import DaftarTugas from "./DaftarTugas";
import CatatanKeuangan from "./keuangan/CatatanKeuangan";

type Halaman = "tugas" | "keuangan";

function App() {
  const [halaman, setHalaman] = useState<Halaman>("tugas");

  return (
    <div className="wadah">
      <div className="tab">
        <button
          className={`tombol-filter ${halaman === "tugas" ? "aktif" : ""}`}
          onClick={() => setHalaman("tugas")}
        >
          Daftar Tugas
        </button>
        <button
          className={`tombol-filter ${halaman === "keuangan" ? "aktif" : ""}`}
          onClick={() => setHalaman("keuangan")}
        >
          Catatan Keuangan
        </button>
      </div>

      {halaman === "tugas" ? <DaftarTugas /> : <CatatanKeuangan />}
    </div>
  );
}

export default App;
```

src/keuangan/ItemTransaksi.tsx – Tugas 3 – komponen 1: satu transaksi

src/keuangan/ItemTransaksi.tsx

```tsx
// src/keuangan/ItemTransaksi.tsx
import { rupiah } from "./format";

export type Transaksi = {
  id: number;
  keterangan: string;
  nominal: number; // sudah berupa angka
  jenis: "pemasukan" | "pengeluaran";
};

type ItemTransaksiProps = {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
};

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const pemasukan = transaksi.jenis === "pemasukan";

  return (
    <li className="item">
      <span>{transaksi.keterangan}</span>

      {/* Pemasukan berwarna hijau, pengeluaran berwarna merah */}
      <span className={pemasukan ? "pemasukan" : "pengeluaran"}>
        {pemasukan ? "+" : "-"} {rupiah(transaksi.nominal)}
      </span>

      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

src/keuangan/format.ts – Tugas 3 – fungsi bantu format rupiah

src/keuangan/format.ts

```ts
// src/keuangan/format.ts
// Mengubah angka menjadi tulisan rupiah, contoh: 50000 -> "Rp 50.000"
export const rupiah = (angka: number) => "Rp " + angka.toLocaleString("id-ID");
```

src/keuangan/FormTransaksi.tsx – Tugas 3 – komponen 2: formulir transaksi

src/keuangan/FormTransaksi.tsx (bagian 1/2)

```tsx
// src/keuangan/FormTransaksi.tsx
import { useState } from "react";
import type { FormEvent } from "react";
import type { Transaksi } from "./ItemTransaksi";

type DataBaru = Omit<Transaksi, "id">; // transaksi tanpa id (id dibuat di induk)

type FormTransaksiProps = {
  onTambah: (data: DataBaru) => void;
};

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  // Tiga input terkendali: nilainya selalu sama dengan state
  const [keterangan, setKeterangan] = useState("");
  const [nominal, setNominal] = useState(""); // input selalu berupa teks
  const [jenis, setJenis] = useState<Transaksi["jenis"]>("pemasukan");

  // Number(...) mengubah teks menjadi angka sebelum dipakai
  const angka = Number(nominal);
  const tidakValid = keterangan.trim() === "" || nominal === "" || !(angka > 0);

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    if (tidakValid) return;

    onTambah({ keterangan: keterangan.trim(), nominal: angka, jenis });
    setKeterangan("");
    setNominal("");
  };

  return (
    <form onSubmit={tanganiKirim} className="form-kolom">
      <input
        className="input"
        type="text"
        placeholder="Keterangan (contoh: Uang saku)"
        value={keterangan}
        onChange={(e) => setKeterangan(e.target.value)}
      />
      <div className="baris">
        <input
          className="input"
          type="number"
          placeholder="Nominal"
          value={nominal}
          onChange={(e) => setNominal(e.target.value)}
        />
        <select
          className="pilih"
          value={jenis}
          onChange={(e) => setJenis(e.target.value as Transaksi["jenis"])}
        >
          <option value="pemasukan">Pemasukan</option>
          <option value="pengeluaran">Pengeluaran</option>
        </select>
      </div>
      <button className="tombol" type="submit" disabled={tidakValid}>
        Tambah Transaksi
      </button>
      {nominal !== "" && !(angka > 0) && (
```

src/keuangan/FormTransaksi.tsx (bagian 2/2)

```tsx
        <p className="galat">Nominal harus lebih besar dari 0.</p>
      )}
    </form>
  );
}

export default FormTransaksi;
```

src/keuangan/CatatanKeuangan.tsx – Tugas 3 – komponen 3 (induk): saldo, daftar, localStorage (memuat gaya tema hijau sendiri)

src/keuangan/CatatanKeuangan.tsx (bagian 1/4)

```tsx
// src/keuangan/CatatanKeuangan.tsx (komponen induk Catatan Keuangan Mini)
import { useState, useEffect } from "react";
import Kartu from "../Kartu";
import FormTransaksi from "./FormTransaksi";
import ItemTransaksi from "./ItemTransaksi";
import { rupiah } from "./format";
import type { Transaksi } from "./ItemTransaksi";

function CatatanKeuangan() {
  // State utama: ambil dari localStorage jika ada
  const [daftar, setDaftar] = useState<Transaksi[]>(() => {
    try {
      const data = localStorage.getItem("catatan-keuangan");
      if (data) return JSON.parse(data) as Transaksi[];
    } catch {
      // isi localStorage rusak: mulai dari kosong
    }
    return [];
  });

  // Efek: simpan ke localStorage setiap daftar berubah
  useEffect(() => {
    localStorage.setItem("catatan-keuangan", JSON.stringify(daftar));
  }, [daftar]);

  const tambah = (data: Omit<Transaksi, "id">) => {
    setDaftar([...daftar, { id: Date.now(), ...data }]); // array baru
  };

  const hapus = (id: number) => {
    setDaftar(daftar.filter((t) => t.id !== id)); // array baru tanpa item itu
  };

  // Nilai turunan (dihitung dari state, bukan disimpan sebagai state baru)
  const totalMasuk = daftar
    .filter((t) => t.jenis === "pemasukan")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const totalKeluar = daftar
    .filter((t) => t.jenis === "pengeluaran")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const saldo = totalMasuk - totalKeluar;

  return (
    <main className="catatan-keuangan">
      <style>{`
        .catatan-keuangan {
          box-sizing: border-box;
          min-height: 100vh;
          padding: clamp(24px, 5vw, 56px) 20px;
          background: linear-gradient(145deg, #ecfdf5 0%, #f7fff9 48%, #d1fae5 100%);
          color: #173b2c;
          font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
serif;
        }
        .catatan-keuangan > * { box-sizing: border-box; }
        .catatan-keuangan > .judul,
        .catatan-keuangan > .subjudul,
        .catatan-keuangan > div {
          width: min(100%, 760px);
          margin-left: auto;
          margin-right: auto;
```

src/keuangan/CatatanKeuangan.tsx (bagian 2/4)

```
        }
        .catatan-keuangan > .judul {
          margin-top: 0;
          margin-bottom: 8px;
          color: #065f46;
          font-size: clamp(2rem, 5vw, 2.8rem);
          font-weight: 800;
          letter-spacing: -0.045em;
        }
        .catatan-keuangan > .subjudul {
          margin-top: 0;
          margin-bottom: 28px;
          color: #527465;
          line-height: 1.6;
        }
        .catatan-keuangan > div {
          margin-bottom: 18px;
          padding: 22px 24px;
          border: 1px solid #ccebd9;
          border-radius: 20px;
          background: rgba(255, 255, 255, 0.9);
          box-shadow: 0 10px 28px rgba(6, 95, 70, 0.08);
        }
        .catatan-keuangan h2, .catatan-keuangan h3 {
          color: #086c4e;
        }
        .catatan-keuangan .saldo {
          margin: 8px 0 18px;
          color: #047857;
          font-size: clamp(2rem, 6vw, 2.7rem);
          font-weight: 800;
          letter-spacing: -0.04em;
        }
        .catatan-keuangan .rekap {
          display: flex;
          flex-wrap: wrap;
          gap: 12px;
        }
        .catatan-keuangan .rekap span {
          padding: 9px 13px;
          border-radius: 999px;
          background: #ecfdf5;
          font-size: 0.92rem;
          font-weight: 650;
        }
        .catatan-keuangan .pemasukan { color: #047857; }
        .catatan-keuangan .pengeluaran { color: #b45309; }
        .catatan-keuangan .kosong {
          margin: 0;
          padding: 20px;
          border-radius: 14px;
          background: #f0fdf4;
          color: #527465;
          text-align: center;
        }
        .catatan-keuangan .daftar {
          display: grid;
          gap: 12px;
          margin: 0;
          padding: 0;
```

src/keuangan/CatatanKeuangan.tsx (bagian 3/4)

```
          list-style: none;
        }
        .catatan-keuangan input,
        .catatan-keuangan select {
          box-sizing: border-box;
          width: 100%;
          padding: 11px 13px;
          border: 1px solid #b7ddc7;
          border-radius: 10px;
          background: #fbfffc;
          color: #173b2c;
          font: inherit;
        }
        .catatan-keuangan input:focus,
        .catatan-keuangan select:focus {
          border-color: #10b981;
          outline: 3px solid rgba(16, 185, 129, 0.16);
        }
        .catatan-keuangan button {
          padding: 11px 16px;
          border: 0;
          border-radius: 10px;
          background: #059669;
          color: white;
          font: inherit;
          font-weight: 700;
          cursor: pointer;
          transition: background 150ms ease, transform 150ms ease;
        }
        .catatan-keuangan button:hover { background: #047857; transform: translateY(-1px); }
        @media (max-width: 520px) {
          .catatan-keuangan > div { padding: 18px; border-radius: 16px; }
        }
      `}</style>
      <h1 className="judul">Catatan Keuangan Mini</h1>
      <p className="subjudul">Laboratorium Informatika - Modul 2: Tugas Terstruktur</p>

      <Kartu judul="Saldo Saat Ini">
        <p className={`saldo ${saldo < 0 ? "pengeluaran" : "pemasukan"}`}>{rupiah(saldo)}</p>
        <div className="rekap">
          <span className="pemasukan">Masuk: {rupiah(totalMasuk)}</span>
          <span className="pengeluaran">Keluar: {rupiah(totalKeluar)}</span>
        </div>
      </Kartu>

      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambah} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {daftar.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Yuk catat transaksi pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftar.map((t) => (
              <ItemTransaksi key={t.id} transaksi={t} onHapus={hapus} />
            ))}
          </ul>
        )}
      </Kartu>
```

src/keuangan/CatatanKeuangan.tsx (bagian 4/4)

```
    </main>
  );
}

export default CatatanKeuangan;
```

Bagian 2. Penjelasan Kode

Ringkasan: aplikasi dipecah menjadi komponen kecil. Data (state) disimpan di komponen induk, lalu dikirim ke anak lewat props; anak "melapor" ke induk dengan memanggil fungsi yang dititipkan lewat props (lifting state up).

Kartu.tsx

- Menerima judul dan children. children berisi apa pun yang ditaruh di antara <Kartu> dan </Kartu>, jadi satu bingkai bisa dipakai ulang.

Penghitung.tsx (Langkah 3)

- Versi keliru memakai let jumlah = 0. Nilainya naik di console, tetapi layar tetap 0 karena React tidak diberi sinyal dan nilainya kembali 0 setiap komponen digambar ulang.
- Versi benar: const [jumlah, setJumlah] = useState(0). Setiap setJumlah dipanggil, React menyimpan nilai baru lalu menggambar ulang layar.
- onClick={tambah} ditulis tanpa tanda kurung agar fungsi dijalankan nanti saat diklik, bukan saat render. {jumlah >= 5 && ... } adalah conditional rendering dengan &&.

FormTugas.tsx (Langkah 4)

- Formulir terkendali: value={teks} + onChange={(e) => setTeks(e.target.value)} membuat isi input selalu sama dengan state.
- Validasi: tombol disabled jika input kosong atau lebih dari 50 karakter; pesan galat muncul dengan &&.
- event.preventDefault() mencegah halaman reload saat formulir dikirim. Setelah valid, onTambah(teks.trim()) melapor ke induk.

ItemTugas.tsx (Langkah 5)

- Menerima satu objek tugas dan dua fungsi pelapor (onUbahStatus, onHapus). className dinamis: tugas selesai dicoret.
- onClick={() => onHapus(tugas.id)} dibungkus arrow function agar fungsi tidak langsung jalan saat render dan bisa mengirim id.

DaftarTugas.tsx (Langkah 6–7, Tugas 1 & 2)

- State daftarTugas ada di sini. Update selalu immutable: tambah dengan [...daftarTugas, tugasBaru], ubah status dengan .map() + { ...tugas, selesai: !tugas.selesai }, hapus dengan .filter().
- jumlahSelesai adalah nilai turunan, dihitung dari state (tidak disimpan sebagai state baru).
- Langkah 7: state awal dibaca dari localStorage lewat useState(() => ...) yang hanya jalan sekali. Efek A ([]) mencatat log sekali; Efek B ([daftarTugas]) mengubah judul tab; Efek C ([daftarTugas]) menyimpan ke localStorage dengan JSON.stringify (dibaca kembali dengan JSON.parse).
- Tugas 1: state filter ("semua" | "belum" | "selesai"). Daftar disaring dengan .filter() sebelum .map(), dan tombol aktif memakai className dinamis (aktif).
- Tugas 2: tombol "Hapus Semua yang Selesai" hanya muncul dengan {jumlahSelesai > 0 && ...}; saat diklik, daftar diganti hasil filter tugas yang belum selesai.

• Tampilan: komponen dibungkus <main className="daftar-tugas-app"> dan memuat elemen <style> berisi gaya tema hijau khusus halaman ini; Kartu dibungkus <section className="panel-tugas">. Hanya tampilan yang berubah, logika state, efek, dan fungsi tetap sama.

Catatan Keuangan Mini (Tugas 3)

• Tiga komponen: CatatanKeuangan (induk, menyimpan state transaksi), FormTransaksi (anak, formulir terkendali), ItemTransaksi (anak, satu baris). Kartu dipakai ulang.

• Nominal dari input berupa teks, jadi diubah dengan Number(nominal) sebelum dijumlahkan (agar "5000" + "2000" tidak menjadi "50002000"). Tombol nonaktif jika keterangan kosong atau nominal bukan angka > 0.

• Saldo = total pemasukan − total pengeluaran, dihitung dengan .filter() dan .reduce() dari state (nilai turunan). Pemasukan hijau dan pengeluaran oranye tua dibedakan lewat className dinamis (pemasukan / pengeluaran). Pesan khusus tampil jika belum ada transaksi.

• Data disimpan ke localStorage dengan useEffect([daftar]) dan dibaca kembali saat aplikasi dibuka.

• Tampilan: komponen dibungkus <main className="catatan-keuangan"> dengan elemen <style> bertema hijau. Karena gaya itu membuat input dan select selebar 100%, ditambahkan dua aturan di akhir App.css agar kolom Nominal tidak mengecil.

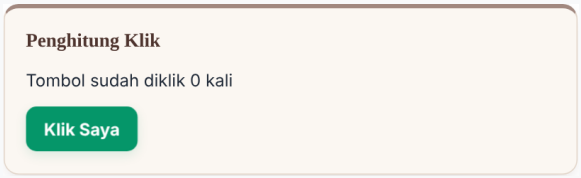
App.tsx

• Kerangka kecil dengan state halaman untuk berpindah antara Daftar Tugas dan Catatan Keuangan.

Bagian 3. Screenshot Hasil Antarmuka

Semua tangkapan layar diambil dari peramban saat npm run dev berjalan (http://127.0.0.1:5174/).

Langkah 3a – Versi keliru (variabel biasa)

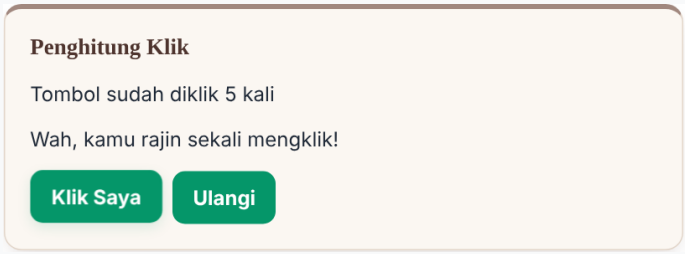


Layar tetap 0 setelah 3 kali klik



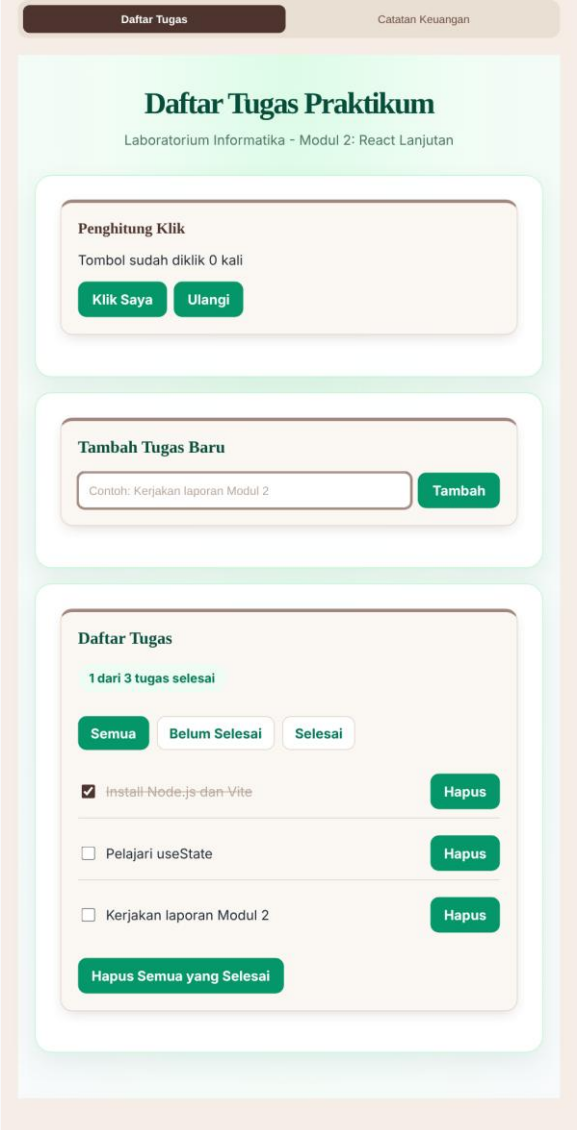
Console: angka naik 1, 2, 3

Langkah 3b – Versi benar (useState)

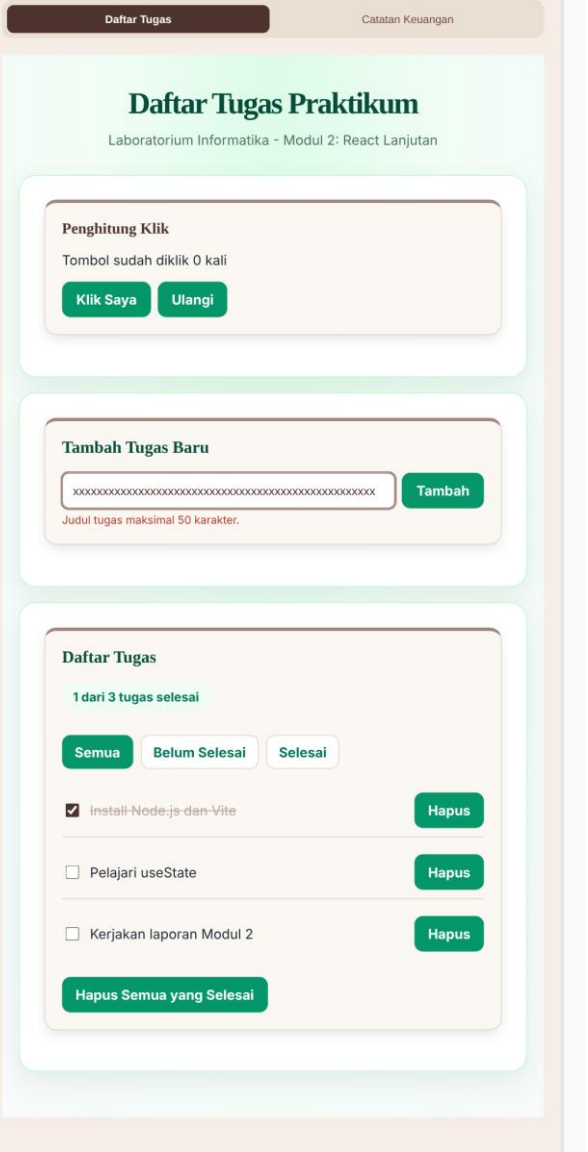


Setelah 5 kali klik: angka bertambah dan pesan muncul

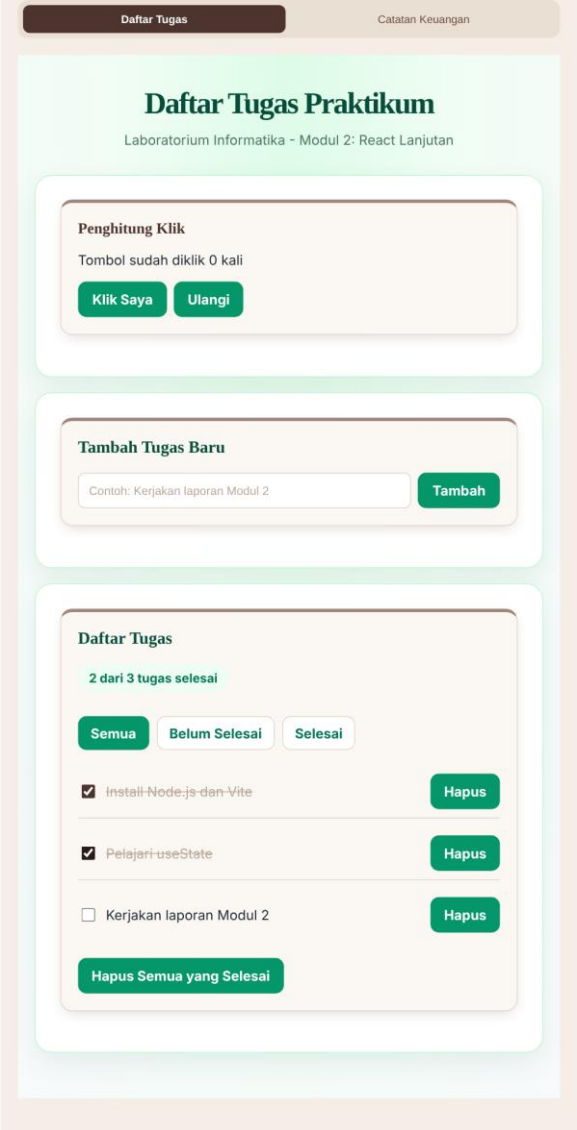
Langkah 6 – Skenario uji 1–5



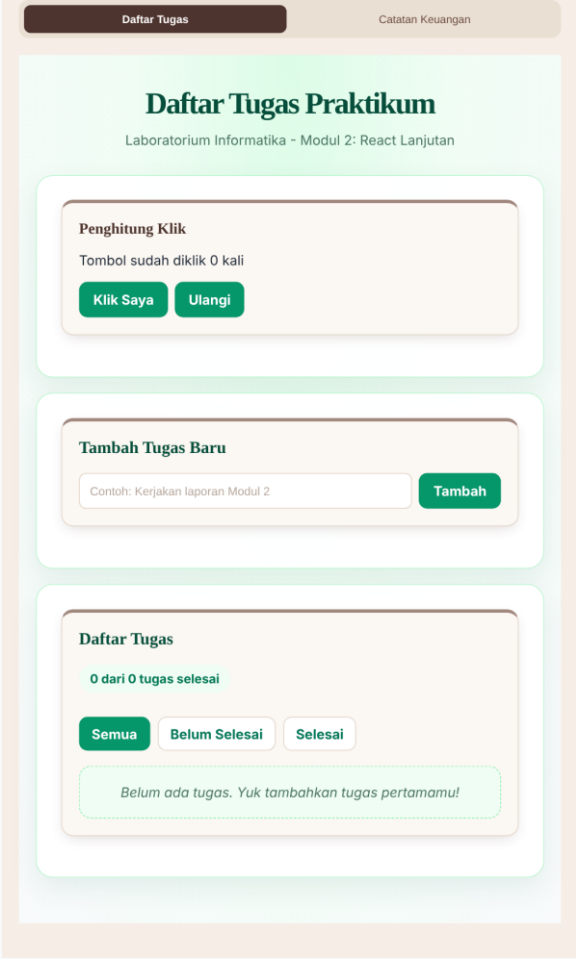
Skenario 1: tugas baru ditambahkan



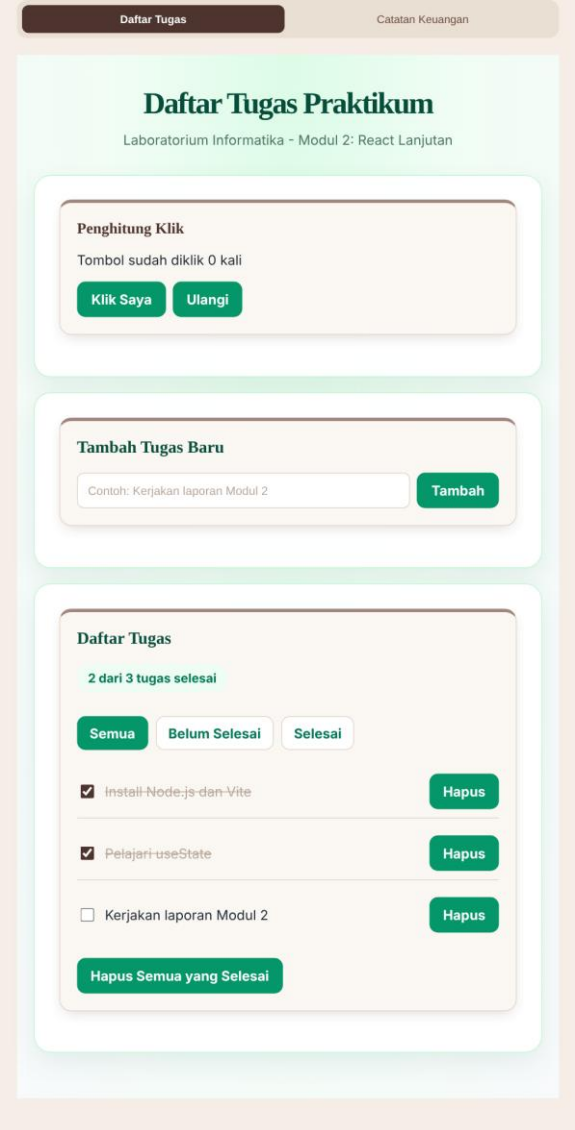
Skenario 2: 51 karakter, galat + tombol nonaktif



Skenario 3: tugas dicentang, dicoret, ringkasan bertambah



Skenario 4: semua tugas dihapus



Skenario 5: setelah refresh (F5) data tetap ada (setelah Langkah 7)

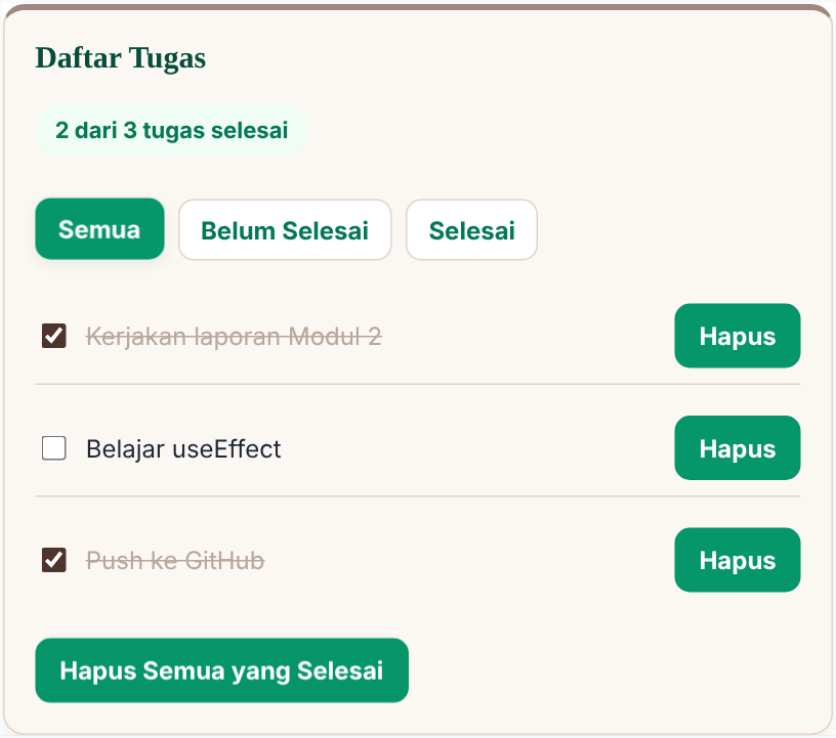
Langkah 7 – Verifikasi useEffect dan localStorage

Verifikasi Langkah 7 (data diambil dari peramban)

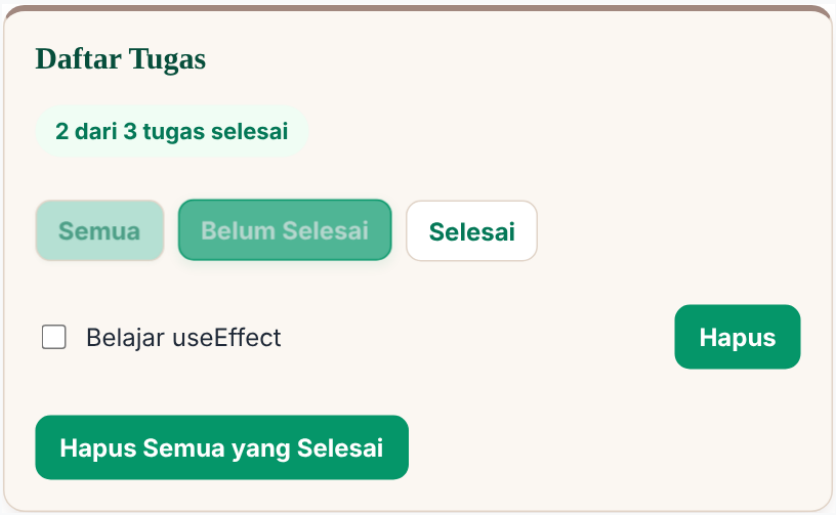
```
Judul tab peramban (document.title): (1) Tugas Belum Selesai
Item tugas setelah refresh (F5): Install Node.js dan Vite | Pelajari useState | Kerjakan laporan Modul 2
DevTools > Application > Local Storage > http://127.0.0.1:5176
kunci : daftar-tugas
nilai : [{"id":1,"judul":"Install Node.js dan Vite","selesai":true},{"id":2,"judul":"Pelajari useState","selesai":true},
{"id":1794201761645,"judul":"Kerjakan laporan Modul 2","selesai":false}]
Console: 'Aplikasi Daftar Tugas siap digunakan!' tampil 2 kali (StrictMode)
```

Judul tab, isi Local Storage, dan log console (diambil dari peramban)

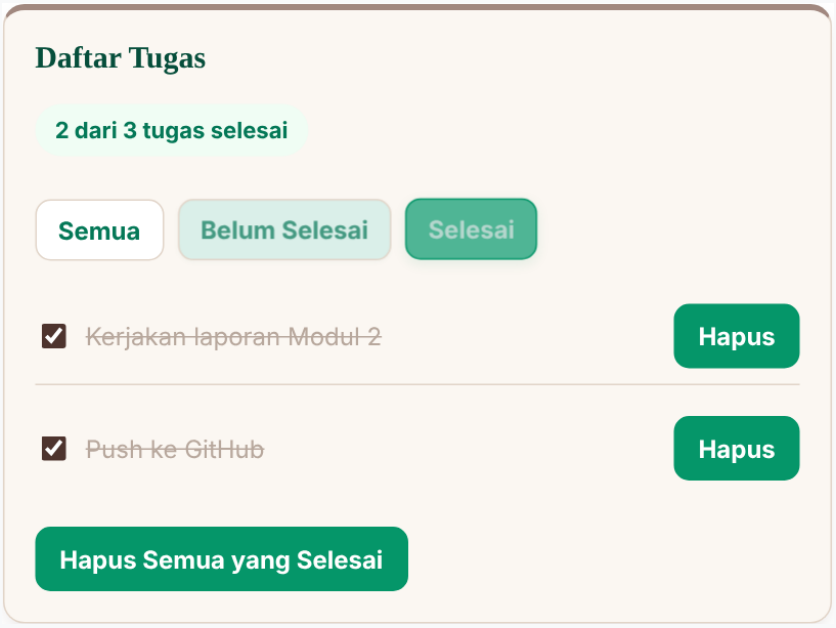
Tugas 1 – Filter



Filter "Semua"

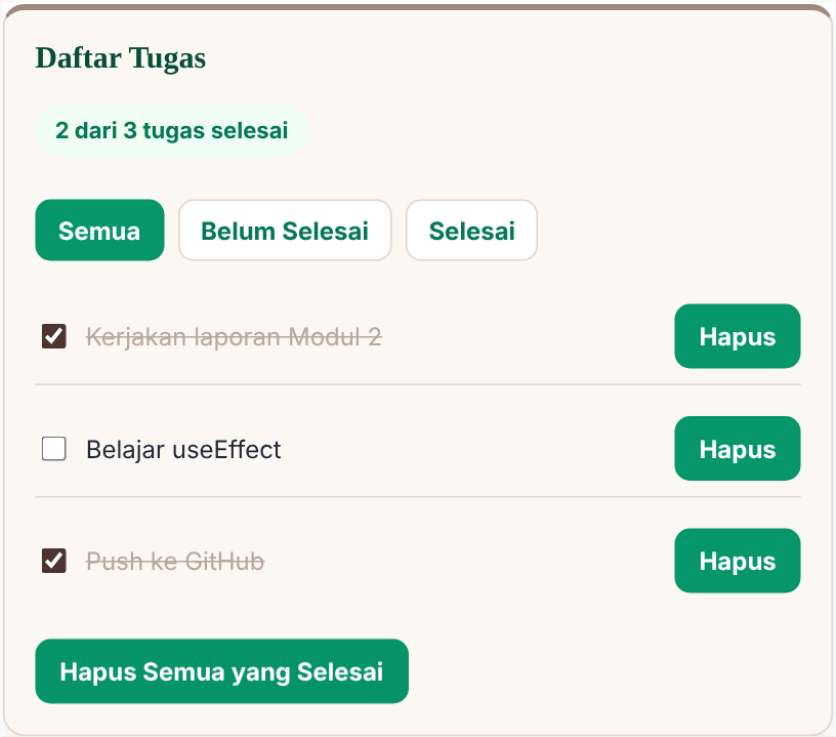


Filter "Belum Selesai"

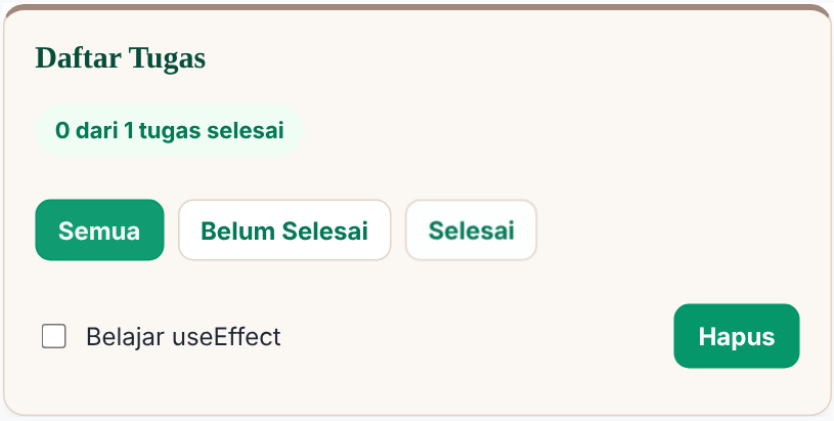


Filter "Selesai"

Tugas 2 – Hapus Semua yang Selesai



Sebelum: ada 2 tugas selesai, tombol muncul

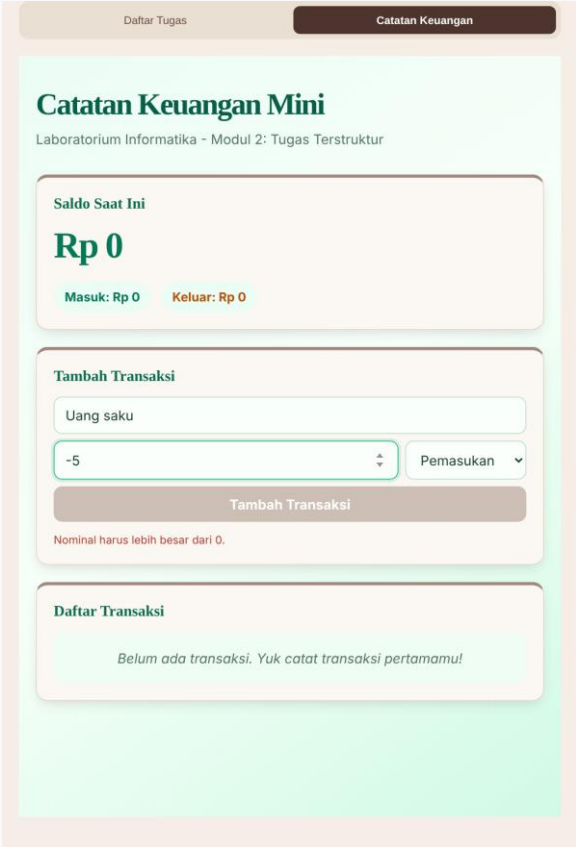


Sesudah diklik: tugas selesai terhapus, tombol hilang

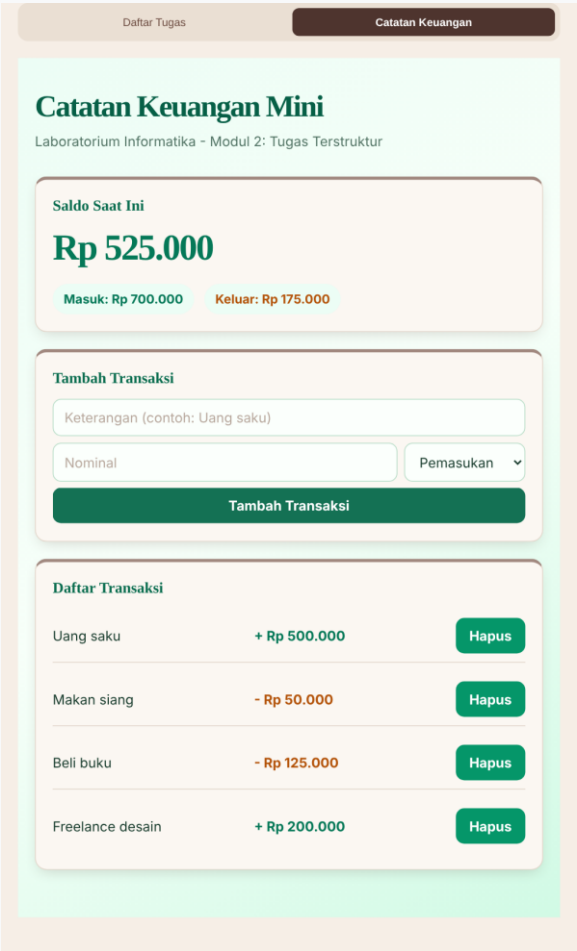
Tugas 3 – Catatan Keuangan Mini



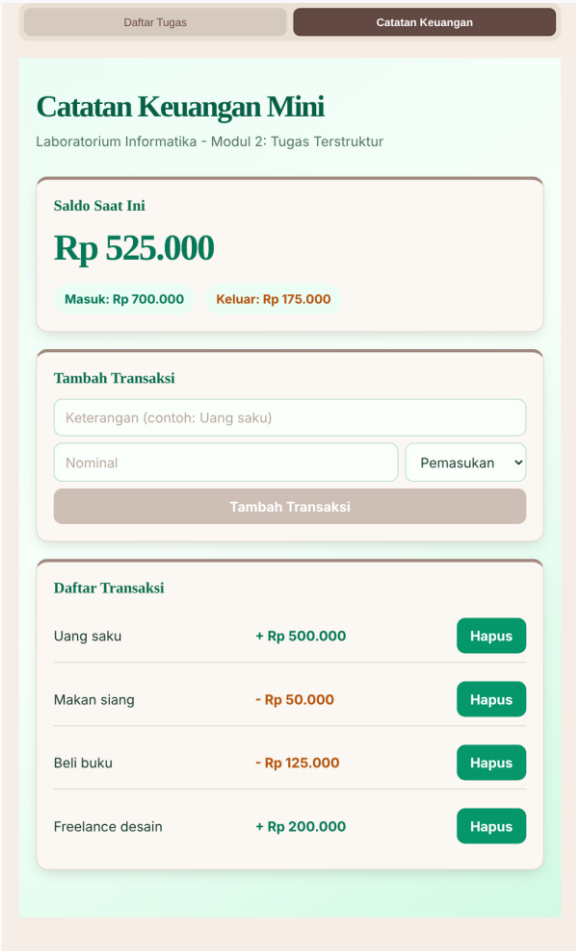
Belum ada transaksi: pesan khusus



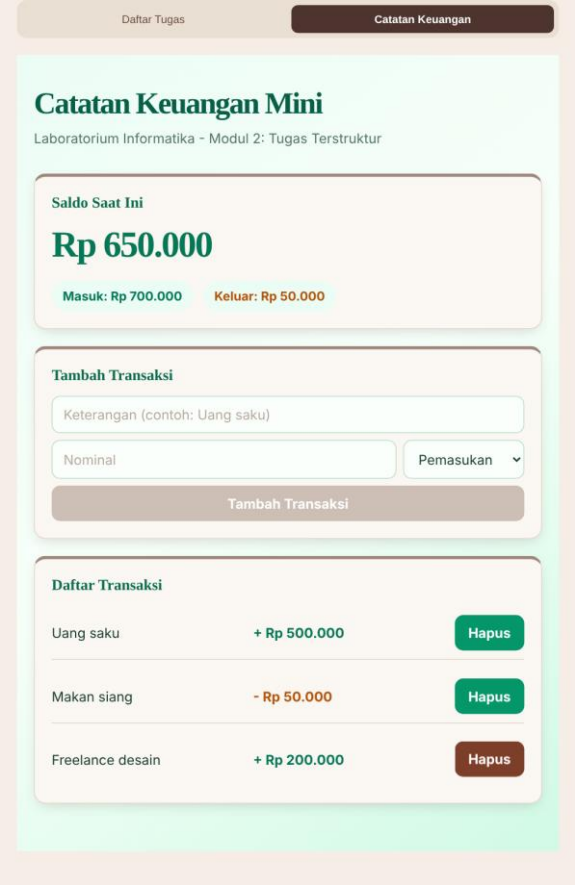
Nominal -5: galat, tombol nonaktif



4 transaksi: saldo Rp 525.000



Setelah refresh data tetap ada



Setelah menghapus "Beli buku": saldo menjadi Rp 650.000

Bagian 4. Penjelasan Hasil Antarmuka

Hasil pengujian dan analisis:

Langkah 3 – Mengapa variabel biasa gagal?

- Pada versi keliru, console menampilkan 1, 2, 3 tetapi layar tetap "0 kali". Nilai let jumlah memang berubah di memori, namun React tidak tahu sehingga tidak menggambar ulang layar. Dan jika komponen digambar ulang, let jumlah = 0 dijalankan lagi dari awal.
- Pada versi benar (useState), setJumlah memberi sinyal ke React: nilai disimpan di antara render dan layar diperbarui. Pesan "Wah, kamu rajin sekali mengklik!" muncul tepat saat jumlah mencapai 5.

Langkah 6 – Skenario uji

- 1. Mengetik judul lalu Enter: tugas baru muncul di daftar dan input kembali kosong (sesuai harapan).
- 2. Mengetik 51 karakter: pesan galat merah muncul dan tombol Tambah nonaktif (sesuai harapan).

• 3. Mencentang tugas: teks tercoret dan ringkasan berubah menjadi "2 dari 3 tugas selesai" (sesuai harapan).

• 4. Menghapus semua tugas: muncul "Belum ada tugas. Yuk tambahkan tugas pertamamu!" (sesuai harapan).

• 5. Refresh (F5): diuji setelah Langkah 7, data tetap ada. Tanpa Langkah 7, data akan kembali ke 2 tugas awal karena state hanya hidup di memori.

Langkah 7 – useEffect

• Judul tab berubah mengikuti jumlah tugas belum selesai, contoh "(1) Tugas Belum Selesai". Data tersimpan di Local Storage kunci daftar-tugas dalam format JSON dan tetap ada setelah refresh.

• Pesan dari Efek A tampil 2 kali di console. Ini normal: <React.StrictMode> sengaja menjalankan efek dua kali selama pengembangan; di versi produksi hanya sekali.

Tugas 1 dan 2

• Filter bekerja: "Semua" menampilkan 3 tugas, "Belum Selesai" hanya yang belum dicentang, "Selesai" hanya yang sudah. Tombol filter yang aktif berwarna hijau (berbeda dari tombol lain).

• Tombol "Hapus Semua yang Selesai" hanya muncul jika ada minimal satu tugas selesai. Setelah diklik, 2 tugas selesai terhapus, sisa 1 tugas, dan tombol menghilang karena kondisinya tidak terpenuhi lagi.

Tugas 3 – Catatan Keuangan Mini

• Awalnya tampil pesan khusus "Belum ada transaksi". Saat nominal -5 dimasukkan, muncul galat dan tombol nonaktif.

• Setelah 4 transaksi (Uang saku +500.000, Makan siang −50.000, Beli buku −125.000, Freelance desain +200.000): masuk Rp 700.000, keluar Rp 175.000, saldo Rp 525.000 (700.000 − 175.000). Pemasukan berwarna hijau dan pengeluaran berwarna oranye tua (#b45309).

• Setelah menghapus "Beli buku" (125.000), saldo naik menjadi Rp 650.000. Data tetap ada setelah refresh karena disimpan di localStorage lewat useEffect.

Kesimpulan

Seluruh langkah dan tugas berjalan sesuai harapan. State diperbarui secara immutable, input terkendali dengan validasi, komponen anak melapor lewat function props, dan useEffect dipakai hanya untuk sinkronisasi ke luar React (judul tab dan localStorage). Build dan lint tanpa error, serta console peramban tanpa galat atau peringatan.